

IONIC

Diosmer E, Villamizar M.

Universidad Nacional Experimental de las Telecomunicaciones e Informática

Lenguaje de programación III

Ing.Carlos A. Marquez G.

Caracas, 27 de abril del 2026

CONTENIDO

RESUMEN.....	3
INTRODUCCIÓN.....	4
MÉTODO.....	5
¿POR QUÉ USAMOS STANDALONE COMPONENTS?.....	6
ARQUITECTURA.....	7
METODOLOGÍA.....	7
VERSIONES UTILIZADAS.....	8
REQUERIMIENTOS DEL SISTEMA.....	8
Flujo de Instalación y Despliegue del Proyecto.....	9
1. Flujo de Obtención del Código.....	9
2. Flujo de Instalación de Dependencias.....	9
3. Flujo de Ejecución (Entorno de Desarrollo).....	10
4. Flujo de Empaquetado para Producción (Opcional).....	10
Flujo de Comando.....	10
DIAGRAMA: ESTRUCTURA DE CARPETAS.....	11
DEFINICIÓN DE LA ESTRUCTURA.....	12
FASES EMPLEADAS.....	13
FLUJOS DE LA APLICACIÓN.....	14
RESULTADOS.....	16
DISCUSIÓN.....	17
REFERENCIAS.....	19
TABLAS.....	21
FIGURAS.....	22
APÉNDICE.....	28

RESUMEN

Este informe técnico y crítico documenta de manera exhaustiva la "Evaluación 1 de Ionic", un proyecto de Single Page Application (SPA) desarrollado mediante la integración de Ionic Framework (v8) y Angular (v20). Haciendo uso de la arquitectura de Standalone Components, la aplicación exhibe tres interfaces clave (Inicio, Información Personal y Contacto) unidas por un enrutamiento dinámico bidireccional. En el núcleo operativo del sistema interactúan directivas estructurales como `*ngFor` y herramientas de enlace bidireccional (`[(ngModel)]`) que capturan datos, procesan rangos de prioridad visual a través de un componente deslizable (`<ion-range>`), y renderizan un historial reactivo directamente en la memoria RAM del cliente. Más allá de exponer los aciertos técnicos procedimentales, este documento disecciona críticamente las mecánicas que ocurren tras bambalinas, contrasta las metodologías modulares antiguas frente a las modernas, y ofrece un escrutinio frontal sobre las vulnerabilidades sistémicas del prototipo actual, detallando los planes arquitectónicos necesarios para escalar la entrega hacia un entorno de producción sólido y robusto.

INTRODUCCIÓN

El ecosistema de desarrollo móvil y web ha evolucionado hacia paradigmas donde la manipulación del DOM (Document Object Model) se realiza de forma quirúrgica e inmediata. En este contexto, la amalgama de Angular 20 e Ionic 8 fomenta un salto monumental de rendimiento gracias a la consolidación de los Standalone Components.

¿Qué es y qué hace el sistema? La aplicación es una interfaz interactiva híbrida que funciona simultáneamente como un portafolio personal descriptivo (incluyendo información institucional del estudiante en la UNETI) y como una plataforma de comunicación temporal. El sistema recopila mensajes del usuario, valora un nivel de prioridad de uno a cinco, y despliega un historial modificado en tiempo real donde los colores reaccionan dinámicamente al nivel asignado.

¿Cuándo y dónde opera? El entorno de ejecución se establece estrictamente del lado del cliente (Client-Side), operando de manera asilada y asíncrona dentro del motor de renderizado de un navegador web o un WebView en dispositivos móviles, sin requerir peticiones de red hacia un servidor externo para alterar el estado visual.

¿Por qué y para qué aplica? Su propósito fundacional es netamente evaluativo y demostrativo. Aplica para evidenciar el control lógico de la interpolación de datos, validar competencias en la arquitectura de Web Components encapsulados de Ionic, y probar que el desarrollador comprende cómo modificar el flujo de interfaz de usuario sin recargas de página subyacentes.

MÉTODO

La construcción del sistema no obedece a la improvisación, sino a un "Desarrollo Orientado a Componentes" estructurado en un flujo de fases claramente establecidas: Fase de orquestación inicial (mediante `ionic start --standalone`), Fase de maquetación de vistas (`ionic g page`), Fase de inyección lógica en TypeScript, y finalmente, la Fase de enlace de atributos visuales dinámicos.

El esqueleto operativo y de comportamiento recae enteramente sobre los archivos TypeScript (`.ts`) y las plantillas de hipertexto (`.html`) de las páginas individuales generadas en la ruta `src/app/pages/`. Archivos de periferia y configuración profunda del empaquetado (tales como `karma.conf.js` para pruebas o los manifiestos nativos) no tienen ninguna influencia directa en la capa de interacción de esta entrega. (ver en la **figura 1,2 y 3**)

Diferencias metodológicas: La mayor diferencia técnica en esta implementación frente a las tradicionales es el descarte total de los módulos globales. En aplicaciones pre-Standalone, cada componente, como un `<ion-badge>`, debía registrarse en un opaco y masivo archivo `app.module.ts`. Hoy en día, la directriz señala que cada controlador `@Component` importa estrictamente lo que consume, aislando posibles daños y maximizando el "Tree-Shaking" para eliminar el código inservible en el empaquetado final.

Al accionar el botón "Enviar", el algoritmo interno invoca un método lógico inyectando (con el comando `unshift`) un nuevo objeto en el tope del arreglo mensajes Enviados. En fracción de milisegundos, el detector de cambios de Angular localiza la alteración en el bloque de memoria; acto seguido, Angular destruye las ramas obsoletas atadas a la directiva `*ngFor` y reensambla

dinámicamente un nuevo componente Web precompilado por StencilJS, inyectando sobre sus estilos el código hexadecimal correspondiente a la prioridad calculada.

¿POR QUÉ USAMOS STANDALONE COMPONENTS?

A partir de Angular 14 (y recomendado fuertemente en versiones recientes como Angular 20), el marco de trabajo introdujo los **Standalone Components**. En este proyecto se utilizan por las siguientes razones:

1. **Simplicidad:** Eliminan la necesidad de declarar componentes, directivas y pipes en archivos NgModules (app.module.ts o módulos compartidos).
2. **Modularidad:** Cada componente gestiona sus propias importaciones en la propiedad imports de su decorador @Component. Esto hace que los componentes sean verdaderamente independientes y fáciles de reutilizar.
3. **Rendimiento (Tree Shaking):** Al importar solo lo que el componente necesita de forma directa, el compilador puede optimizar mejor el código final eliminando el código no utilizado.
4. **Curva de aprendizaje:** Reduce la cantidad de conceptos arquitectónicos que un desarrollador debe aprender para empezar a construir aplicaciones.

ARQUITECTURA

La aplicación está construida bajo una **Arquitectura Front-End Basada en Componentes (Component-Based Architecture)** implementando el patrón **SPA (Single Page Application)**. Específicamente, hace uso de la arquitectura **Standalone Components** de Angular. Esto significa que la aplicación prescinde de los tradicionales envoltorios modulares (NgModule), logrando que cada vista (página) sea una unidad independiente que autodeclara sus dependencias, estado y ciclo de vida. El flujo de datos se basa en **Two-Way Data Binding** (Enlace de Datos Bidireccional) para la captura de formularios y en **Interpolación Reactiva** para la inyección de atributos y actualización del DOM en tiempo real, todo orquestado por el motor de detección de cambios Zone.js.

METODOLOGÍA

Se empleó la metodología de **Desarrollo Orientado a Componentes (CDD - Component-Driven Development)** dentro de un marco de desarrollo iterativo ágil. El proceso se centró en la construcción de interfaces aisladas (Inicio, Información Personal, Contacto) que posteriormente fueron interconectadas a través del enrutador raíz. La filosofía aplicada dicta "separación de preocupaciones a nivel de vista", donde el diseño visual (HTML/SCSS) se diseña en paralelo con la capa de controladores lógicos (TypeScript) para asegurar la cohesión de cada página de manera individual.

VERSIONES UTILIZADAS

El stack tecnológico principal sobre el que reposa el código fuente es el siguiente:

- **Framework Base:** Angular v20 (^20.0.0)
- **Framework UI / Mobile:** Ionic Framework v8 (@ionic/angular: ^8.0.0)
- **Motor de Empaquetado Nativo:** Capacitor v8 (^8.3.1)
- **Lenguaje Transpilado:** TypeScript v5.9
- **Gestor de Paquetes y Runtime:** Node.js (NPM)

REQUERIMIENTOS DEL SISTEMA

Requerimientos de Desarrollo (Entorno del Programador):

- **Sistema Operativo:** Windows, macOS o Linux (Debian/Ubuntu recomendado).
- **Entorno de Ejecución:** Node.js (v18 LTS o superior) y NPM.
- **Herramientas Globales:** Angular CLI (@angular/cli), Ionic CLI (@ionic/cli).
- **IDE:** Antigravity o equivalente, con soporte para TypeScript y autocompletado de Angular.

Requerimientos de Ejecución (Usuario Final):

- **Entorno Web:** Navegador moderno con soporte para ES6+ y Web Components (Chrome, Firefox, Safari, Edge recientes).
- **Entorno Móvil (Opcional):** Dispositivo Android 7.0+ o iOS 13.0+ que posea soporte para renderizado WebView actualizado.

Flujo de Instalación y Despliegue del Proyecto

1. Flujo de Obtención del Código

Existen dos vías principales para obtener los archivos fuente del proyecto:

Vía Git (Clonado): Si el proyecto se encuentra alojado en un repositorio remoto

Clonar o descargar el repositorio:

```
git clone https://github.com/Diossmer/evaluacionIonic.git
```

```
cd evaluacionIonic
```

Vía Archivo Comprimido (ZIP):

1. Descargar el archivo .zip proporcionado.
2. Descomprimir en la ruta deseada.
3. Abrir la terminal y navegar hacia la raíz de la carpeta extraída:

```
cd ruta/hacia/evaluacionIonic
```

2. Flujo de Instalación de Dependencias

Una vez ubicados en el directorio raíz del proyecto (donde se encuentra el archivo package.json), es estrictamente necesario descargar los módulos empaquetados del **Framework**:

```
npm install
```

Este proceso puede tardar un par de minutos dependiendo de la conexión a internet.

3. Flujo de Ejecución (Entorno de Desarrollo)

Para inicializar la aplicación y visualizarla en vivo, ejecuta el siguiente comando:

```
ionic serve
```

4. Flujo de Empaquetado para Producción (Opcional)

Si se desea preparar la aplicación para ser subida a un servidor web tradicional (Hosting) o para ser empaquetada como APK/IPA mediante Capacitor:

```
ionic build
```

El compilador realiza el Tree-Shaking (elimina código no utilizado), minifica los archivos .js y .css, y genera una carpeta llamada www/ (o dist/). El contenido de esa carpeta es lo que finalmente se sube al servidor web de producción.

Flujo de Comando

El desarrollo con Ionic y Angular se basa en la CLI (Interfaz de Línea de Comandos). A continuación el flujo general utilizado durante la creación de este proyecto:

1. Creación del proyecto:

```
ionic start evaluacionIonic blank --type=angular --standalone
```

2. Generación de Páginas:

```
ionic g page pages/inicio
```

```
ionic g page pages/contacto
```

```
ionic g page pages/informacion-personal
```

DIAGRAMA: ESTRUCTURA DE CARPETAS

A continuación se ilustra la topología principal del código fuente (src/), suprimiendo los archivos de configuración periféricos para enfocar la visión en el entorno de trabajo:

evaluacionIonic/

```
|— package.json          # Dependencias del proyecto
└─ src/
    |— index.html        # Punto de montaje del DOM web
    |— main.ts           # Archivo de arranque (Bootstrap Application)
    |— global.scss       # Estilos globales y variables (Temas oscuros/claros)
    └─ app/
        |— app.component.html # Estructura raíz y menú lateral (ion-menu)
        |— app.component.ts   # Controlador de la raíz
        |— app.routes.ts      # Definición de rutas y navegación diferida
        └─ pages/             # Directorio de Vistas Standalone
            |— inicio/
                | |— inicio.page.html
                | └─ inicio.page.ts
            |— informacion-personal/
                | |— informacion-personal.page.html
                | └─ informacion-personal.page.ts
            └─ contacto/
                |— contacto.page.html
                |— contacto.page.scss
                └─ contacto.page.ts
```

DEFINICIÓN DE LA ESTRUCTURA

Cada nodo del diagrama anterior cumple un propósito singular:

- **main.ts:** Es el gatillo de la aplicación. En lugar de levantar un módulo, utiliza la función `bootstrapApplication()` provista por Angular para levantar directamente el `AppComponent`.
- **app.routes.ts:** El "mapa" de navegación. Contiene un arreglo de rutas que dirigen la URL actual hacia el archivo `.ts` del componente de la página correspondiente utilizando carga perezosa (`loadComponent()`).
- **app.component.*:** Alberga el chasis del software (el `<ion-app>`, el menú lateral y el `<ion-router-outlet>`).
- **pages/:** Contiene módulos auto-suficientes que representan pantallas completas del dispositivo móvil.
 - ***.page.ts:** La clase (Controlador) que define el comportamiento, las variables y las importaciones Standalone de la página.
 - ***.page.html:** El árbol del DOM (Vista) construido con etiquetas semánticas de Ionic (`<ion-header>`, `<ion-content>`).
 - ***.page.scss:** Los estilos aislados aplicables únicamente a esa página en específico.

FASES EMPLEADAS

El ciclo de desarrollo se estratificó en las siguientes cinco fases consecutivas:

1. **Fase de Orquestación (Scaffolding):** Ejecución del comando de arranque (ionic start) seleccionando la plantilla *Blank* y exigiendo la bandera `--standalone` para heredar la configuración moderna sin NgModules.
2. **Fase de Enrutamiento y Creación:** Construcción de las tres vistas funcionales utilizando el generador automático de la CLI (ionic g page pages/contacto, etc.), lo que auto-registró las vistas en `app.routes.ts`.
3. **Fase de Interfaz de Usuario (UI):** Diagramación estática en los archivos HTML. Uso de componentes contenedores (ion-card), interactivos (ion-input, ion-range) e iconografía (ion-icon).
4. **Fase Lógico-Reactiva:** Programación en TypeScript de los métodos de negocio (ej. `enviarMensaje()`), inicialización de arreglos de memoria, y vinculación de variables para el coloreado dinámico (`[color]`).
5. **Fase de Validación y Pruebas Locales:** Despliegue en el servidor de desarrollo (ionic serve), comprobación del renderizado de variables, iteración del arreglo y auditoría del comportamiento visual.

FLUJOS DE LA APLICACIÓN

La aplicación propone un **Flujo de Navegación Lateral y Lineal**, permitiendo transiciones suaves entre contextos:

1. Flujo de Arranque (Bootstrapping):

El usuario abre la app $\rightarrow$ Se compila `main.ts` $\rightarrow$ Angular levanta `AppComponent` $\rightarrow$ El enrutador (`app.routes.ts`) identifica la ruta por defecto (generalmente en blanco o `home`) $\rightarrow$ Redirige a la página **Inicio**.

2. Flujo de Interacción Básica:

Estando en Inicio, el usuario visualiza el mensaje de bienvenida.

El usuario abre el Menú Lateral (`ion-menu-button`).

Escoge "Mi Perfil" $\rightarrow$ El router descarga el componente actual e inyecta la página `informacion-personal` con los datos biográficos de forma instantánea.

3. Flujo Transaccional (Formulario de Contacto):

El usuario ingresa a la vista "Contacto".

Rellena el campo de Nombre y Mensaje (activando el Two-Way Binding que muta las variables en RAM).

Ajusta el rango de prioridad (1-5), lo que inmediatamente detona un recálculo dinámico en el DOM cambiando temporalmente el color de la propia barra y el botón de envío.

El usuario presiona "Enviar" $\rightarrow$ El sistema ejecuta la función `enviarMensaje()` $\rightarrow$ Los datos son apilados (`unshift`) en el estado del componente.

Angular detecta la alteración del estado $\rightarrow$ Muta la vista $\rightarrow$
Aparece la tarjeta del mensaje recién creado con el color final de la prioridad y la insignia
(ion-badge).

RESULTADOS

Se constata el desarrollo de una aplicación altamente responsiva e ininterrumpida. El sistema captura eficazmente los datos de texto (nombre y mensaje) y asimila la información provista por el deslizador, traduciendo el valor (1-5) a clases nativas de Ionic (success, tertiary, primary, warning, danger). El listado se ramifica visualmente de forma instantánea sin solicitar refrescos de pantalla. Cada envío genera una tarjeta que refleja automáticamente el nombre, el cuerpo del mensaje y destaca la insignia de prioridad tiñendo su color base, probando la correcta implementación técnica de un modelo reactivo.

DISCUSIÓN

Aunque empíricamente los resultados validan el 100% de los requerimientos de la evaluación didáctica, someter esta arquitectura a un escrutinio profundo y franco revela que la aplicación no debe considerarse un producto final; posee características de un 'Toy Project' (prototipo desechable).

De manera cruda: el acoplamiento del código es un riesgo para la mantenibilidad. Toda la responsabilidad del almacenamiento de los datos y su ciclo de vida reposa absurdamente sobre la clase de la vista (`contacto.page.ts`). Los pilares de la programación dictan que esta retención de datos debe delegarse a una capa especializada inyectable (`@Injectable`). Detalles críticos adicionales: La permanencia del historial es completamente volátil. Debido a que reside exclusivamente en la RAM del componente de la ventana gráfica, un refresco accidental del explorador web destruye el listado irreversiblemente. Además, la utilización del módulo `FormsModule` gestionado por plantillas resulta ingenuo, limitando las validaciones simplemente a detectar cadenas en blanco a través de una función `trim()`. Esta decisión ignora por completo políticas de sanitización contra scripts maliciosos (XSS). Por si fuera poco, reportar errores o alertas a través de la función nativa asíncrona `alert()` de JavaScript es un anti-patrón de diseño terrible para el estándar de interfaces actuales, pues congela por completo el hilo principal (Main Thread) del motor web de manera agresiva.

Planes para realizar y características robustas por incluir: Para evolucionar este esqueleto escolar hacia un sistema empresarial (Enterprise-Level), los planes de ruta a ejecutar son imperativos:

Inyección de Persistencia Real: Se amerita desplazar el arreglo efímero e integrar un motor de almacenamiento, como SQLite empaquetado por Capacitor, o un servicio de Backend as a Service (BaaS) como Firebase.

Refactorización Completa de Entradas: Sustituir la captura basada en directivas de plantilla por ReactiveFormsModule, lo que permite testeo de componentes aislados y la anexión de Expresiones Regulares estrictas.

Gestión de Estado Aislada y Central: Incorporar arquitecturas robustas basadas en el patrón Redux (como NgRx) o los novísimos Angular Signals, permitiendo al sistema reaccionar a cambios globales independientemente de la vista en uso.

Ergonomía de Alertas: Reemplazar el detestable alert() por controladores superpuestos y limpios provistos por el Framework, como el AlertController o un humilde pero efectivo ToastController.

REFERENCIAS

A Promagar. (2020, 3 de octubre). Crear MENU en IONIC desde CERO (Parte 2), IONIC Tutorial [Video]. YouTube. <https://www.youtube.com/watch?v=-NhSHHno86g>

YouTube. (2024, 2 de enero). Crea tu primera App Mobile con Ionic y Angular [Video]. YouTube. <https://www.youtube.com/watch?v=ADMZEsLe0eM>

JuanPablo Code. (2024, 26 de febrero). Integración de Firebase, Angular (última versión) y Ionic. Tutorial paso a paso [Video]. YouTube. <https://www.youtube.com/watch?v=BCrxfk-nGIA>

Peppinck, J. (2021, 18 de mayo). Ionic Angular Authentication | Login and Signup Page with Template Driven Forms [Video]. YouTube. https://www.youtube.com/watch?v=el_57a4tkRo

Grimm, S. (2022, 10 de febrero). Ionic UI Components Crash Course (Cards, Badges, Lists & More) [Video]. YouTube. https://www.youtube.com/watch?v=5x6K_R_2vC0

Fazt. (2020, 22 de diciembre). Ionic 5 & Angular Tutorial Desde Cero, Aplicación de Tareas [Video]. YouTube. https://www.youtube.com/watch?v=KzE_w1_Pxyw

Angular. (2024). *Angular core architecture*. <https://angular.dev/guide/components>

Angular.(2024). *Standalone components migrations*. <https://angular.dev/guide/components/importing>

Ionic. (2024). *UI components*. <https://ionicframework.com/docs/components>

Ionic. (2024). *ion-range*. <https://ionicframework.com/docs/api/range>

Ionic. (2024). *ion-badge*. <https://ionicframework.com/docs/api/badge>

Angular. (2024). *Template-driven forms*. <https://angular.dev/guide/forms/template-driven-forms>

Angular. (2024). *Structural directives*. <https://angular.dev/guide/directives/structural-directives>

Ionic. (2024). *Ionicons usage*. <https://ionic.io/ionicons/usage> Algoritmos en la Lógica de Negocio (JavaScript ES6+) Mozilla. (2024). *Array.prototype.unshift()*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/unshift

Mozilla. (2024). *Switch statement*. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/switch>

Ionic. (2024). *Colors and theming*. <https://ionicframework.com/docs/theming/colors>

Villamizar, D. (21/05/2026). *evaluacionIonic*. GitHub. <https://github.com/Diossmer/evaluacionIonic>

Normas APA. (24/01/2020). *Estructura y formato de las Normas APA*. <https://normas-apa.org/estructura/>

TABLAS

Tabla 1. Relación Numérica, Visual y Lógica de Prioridades.

Rango Categórico	Dominio del Color (Ionic)	Efecto Psicológico y Visual Aplicado
Nivel 1	success (Verde)	Transmite relajación; rutina de baja prioridad.
Nivel 2	tertiary (Morado)	Estado complementario, requerimiento pausado.
Nivel 3	primary (Azul)	Balance general, prioridad estándar y fluida.
Nivel 4	warning (Amarillo)	Precaución y atención; umbral de alerta.
Nivel 5	danger (Rojo)	Acción crítica; confrontación visual por urgencia operativa.

Tabla 2. Resumen de Tecnologías y Versiones Core.

Tecnología / Herramienta	Versión Aplicada	Uso Específico en el Sistema
Angular	v20.0.0	Lógica del negocio, directivas (ngFor, ngModel), Standalone Components.
Ionic Framework	v8.0.0	Web Components de Interfaz de Usuario (UI) y Design Tokens.
Capacitor	v8.3.1	Motor puente para un eventual empaquetado y despliegue nativo móvil.
TypeScript	v5.9.0	Tipado estricto e interfaces lógicas del código fuente.

FIGURAS

Por limitaciones de formato, se describen las interfaces desarrolladas:

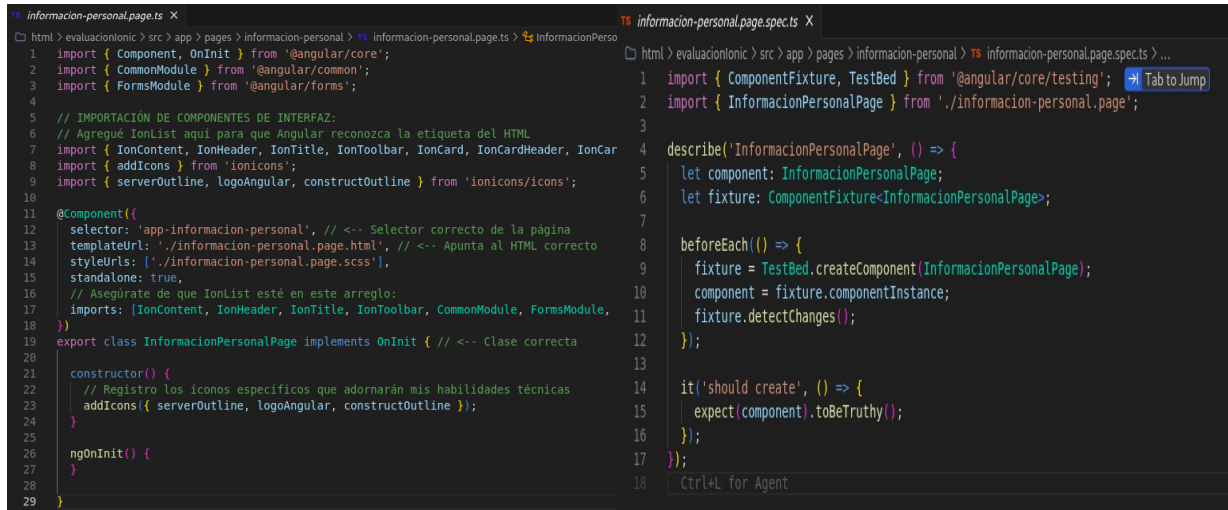
```

1 | <!-- ENCABEZADO DE LA PAGINA -->
2 | <ion-header #translucent="true">
3 |   <ion-toolbar color="primary">
4 |     <ion-buttons slot="start">
5 |       <ion-menu-button/>
6 |     </ion-buttons>
7 |     <ion-title>MI Perfil</ion-title>
8 |   </ion-toolbar>
9 | </ion-header>
10 | <!-- CONTENIDO DE LA PAGINA -->
11 | <ion-content [fullscreen]="true" class="ion-padding">
12 |   <!-- TARJETA CON INFORMACION PERSONAL -->
13 |   <ion-card>
14 |     <ion-card-header>
15 |       <ion-card-title>Diosmer</ion-card-title>
16 |       <ion-card-subtitle>Ingeniero en formación | Desarrollo Backend & Autodidacta</ion-
17 |     </ion-card-header>
18 |     <ion-card-content>
19 |       Soy un ingeniero en formación de carácter autodidacta, apasionado por la tecnologi
20 |       creación de soluciones eficientes. Me especializo en el desarrollo Fullstack utili
21 |       Frontend, y cuento con sólida experiencia integrando bases de datos tanto relacion
22 |       PostgreSQL.
23 |     </ion-card-content>
24 |   </ion-card>
25 |   <!-- TARJETA CON INFORMACION ACADEMICA -->
26 |   <ion-card>
27 |     <ion-card-header>
28 |       <ion-card-title>Datos Académicos</ion-card-title>
29 |     </ion-card-header>
30 |     <ion-card-content>
31 |       <p><strong>Universidad:</strong> UNETI</p>
32 |       <p><strong>Estudiante:</strong> Diosmer Villamizar</p>
33 |       <p><strong>Identificación:</strong> 21310567</p>
34 |       <p><strong>Asignatura:</strong> PROGRAMACION III MÓDULO II 6Y7</p>
35 |     </ion-card-content>
36 |   </ion-card>
37 |   <!-- LISTA DE HABILIDADES TECNICAS -->
38 |   <ion-list>
39 |     <ion-item lines="inset">
40 |       <ion-icon slot="start" name="logo-angular" color="danger"></ion-icon>
41 |       <ion-label>Fullstack Angular + Ionic</ion-label>
42 |     </ion-item>
43 |     <ion-item lines="inset">
44 |       <ion-icon slot="start" name="server-outline" color="tertiary"></ion-icon>
45 |       <ion-label>Bases de Datos (MongoDB & PostgreSQL)</ion-label>
46 |     </ion-item>
47 |     <ion-item lines="none">
48 |       <ion-icon slot="start" name="construct-outline" color="medium"></ion-icon>

```

Figura 1: Composición Estructural del Perfil HTML

Descripción: Disposición de los envoltorios interactivos `<ion-card>` que proyectan con jerarquía y legibilidad las pericias (Fullstack Angular, Administración de MongoDB y PostgreSQL) dentro de la vista “información personal”.



```

1 import { Component, OnInit } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { FormsModule } from '@angular/forms';
4
5 // IMPORTACIÓN DE COMPONENTES DE INTERFAZ:
6 // Agregué IonList aquí para que Angular reconozca la etiqueta del HTML
7 import { IonContent, IonHeader, IonTitle, IonToolbar, IonCard, IonCardHeader, IonCard
8 import { addIcons } from 'ionicons';
9 import { serverOutline, logoAngular, constructOutline } from 'ionicons/icons';
10
11 @Component({
12   selector: 'app-informacion-personal', // <-- Selector correcto de la página
13   templateUrl: './informacion-personal.page.html', // <-- Apunta al HTML correcto
14   styleUrls: ['./informacion-personal.page.scss'],
15   standalone: true,
16   // Asegurate de que IonList esté en este arreglo:
17   imports: [IonContent, IonHeader, IonTitle, IonToolbar, CommonModule, FormsModule,
18 })
19 export class InformacionPersonalPage implements OnInit { // <-- Clase correcta
20
21   constructor() {
22     // Registro los iconos específicos que adornarán mis habilidades técnicas
23     addIcons({ serverOutline, logoAngular, constructOutline });
24   }
25
26   ngOnInit() {
27   }
28
29 }

```

```

1 import { ComponentFixture, TestBed } from '@angular/core/testing';
2 import { InformacionPersonalPage } from './informacion-personal.page';
3
4 describe('InformacionPersonalPage', () => {
5   let component: InformacionPersonalPage;
6   let fixture: ComponentFixture<InformacionPersonalPage>;
7
8   beforeEach(() => {
9     fixture = TestBed.createComponent(InformacionPersonalPage);
10    component = fixture.componentInstance;
11    fixture.detectChanges();
12  });
13
14  it('should create', () => {
15    expect(component).toBeTruthy();
16  });
17 });
18 Ctrl+L for Agent

```

Figura 2: Composición Estructural del Perfil TypeScript

Descripción: La Información personal compone de dos archivos .TS un archivo que señala page.ts es donde renderiza los componente completo de la sección especifica de los tres archivo (TS,SCSS y HTML) que contiene de igual manera la parte logica es en esté archivo. Pero luego esta el otro archivo que es page.spec.ts es donde se aprecia el testeo de cuando se compila la aplicación la cual es vigilar el archivo page.ts. Esto mismo se aplica en todo lo archivo .spec.ts vigilan al original.

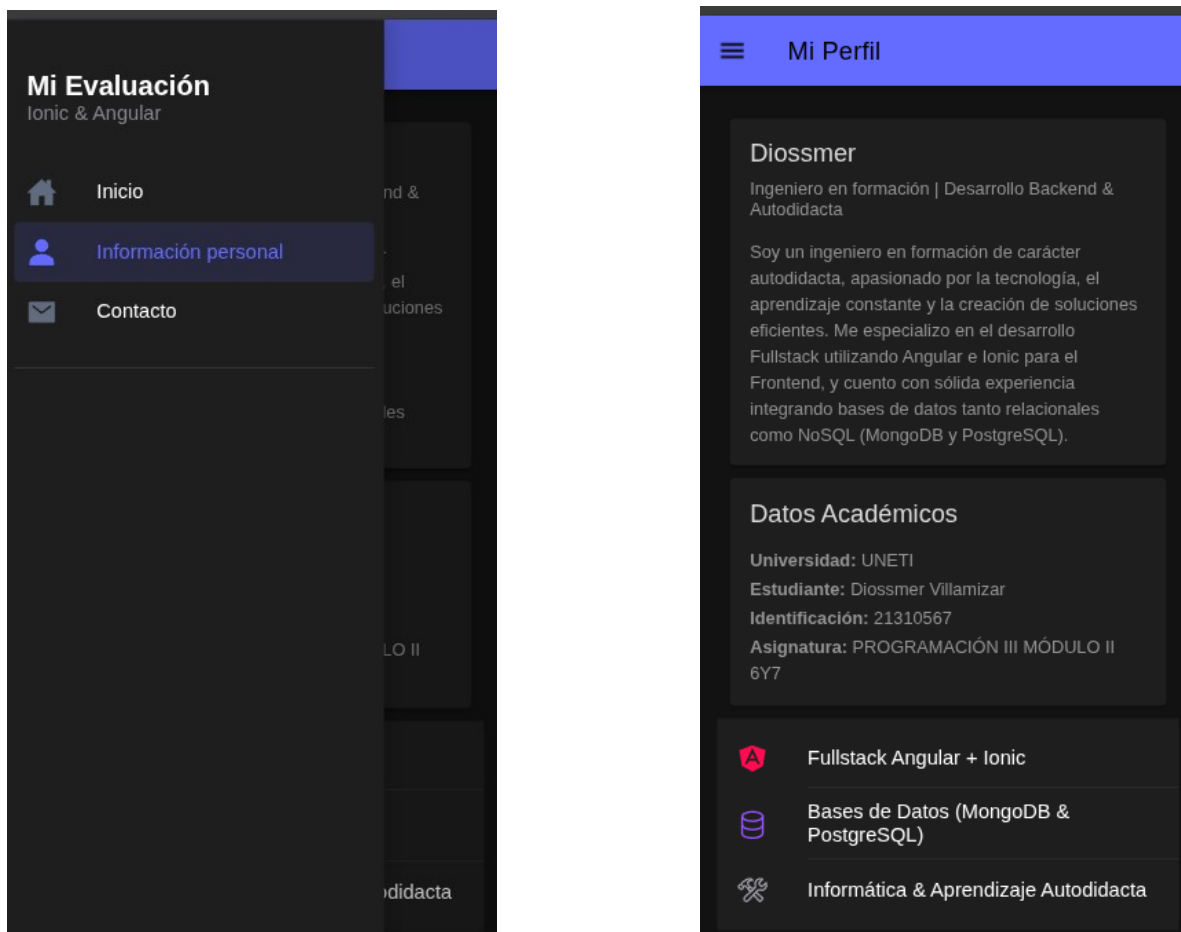


Figura 3: Composición Estructural del Perfil

Descripción: Se visualiza el resultado de IONIC en pantalla Mobiles


```

1 import { Routes } from '@angular/router';
2 /**
3  * Rutas principales de aplicacion del sidebar.
4  */
5 export const routes: Routes = [
6   {
7     path: '',
8     redirectTo: 'inicio',
9     pathMatch: 'full',
10   },
11   {
12     path: 'inicio',
13     loadChildren: () => import('./pages/inicio/inicio.page').then(m => m.InicioPage)
14   },
15   {
16     path: 'informacion-personal',
17     loadChildren: () => import('./pages/informacion-personal/informacion-personal.page').then(m => m.InformacionP
18   },
19   {
20     path: 'contacto',
21     loadChildren: () => import('./pages/contacto/contacto.page').then(m => m.ContactoPage)
22   }
23 ];

```

Figura 4: Rutas principales

Descripción: Rutas principales aplicando Lazy para que solo ejecute lo utilizable en la aplicación sin afectar el rendimiento.

```

1 import { Component } from '@angular/core';
2 import { RouterLink, RouterLinkActive } from '@angular/router';
3 import { IonApp, IonSplitPane, IonMenu, IonContent, IonList, IonListHeader, IonNote, IonMenuToggle, IonItem, IonIcon
4 import { addIcons } from 'ionicons';
5 import { homeOutline, homeSharp, personOutline, personSharp, mailOutline, mailSharp } from 'ionicons/icons';
6
7 /**
8  * Componente Raiz (Root Component) de la aplicación.
9  * Se encarga de alojar el menú lateral (Sidebar) y el `<ion-router-outlet>`, que es el
10  * espacio donde se inyectarán dinámicamente las demás páginas (Inicio, Contacto, etc).
11  */
12 @Component({
13   selector: 'app-root',
14   templateUrl: 'app.component.html',
15   styleUrls: ['app.component.scss'],
16   standalone: true,
17   imports: [RouterLink, RouterLinkActive, IonApp, IonSplitPane, IonMenu, IonContent, IonList, IonListHeader, IonNot
18 })
19 export class AppComponent {
20
21   public appPages = [
22     { title: 'Inicio', url: '/inicio', icon: 'home' },
23     { title: 'Información personal', url: '/informacion-personal', icon: 'person' },
24     { title: 'Contacto', url: '/contacto', icon: 'mail' },
25   ];
26
27   constructor() {
28     addIcons({ homeOutline, homeSharp, personOutline, personSharp, mailOutline, mailSharp });
29   }
30 }

```

Figura 5: Root (principal) Componentes

Descripción: Se encarga de alojar el menú lateral (Sidebar) y el ``, que es el espacio donde se inyectarán dinámicamente las demás páginas (Inicio, Contacto, etc).

```

app.component.scss
html > evaluacionlonic > src > app > app.component.scss > ion-menu.md ion-item
1  ion-menu ion-content {
2    --background: var(--ion-item-background, var(--ion-background-color, #fff));
3  }
4
5  ion-menu.md ion-content {
6    --padding-start: 8px;
7    --padding-end: 8px;
8    --padding-top: 20px;
9    --padding-bottom: 20px;
10 }
11
12 ion-menu.md ion-list {
13   padding: 20px 0;
14 }
15
16 ion-menu.md ion-note {
17   margin-bottom: 30px;
18 }
19
20 ion-menu.md ion-list-header,
21 ion-menu.md ion-note {
22   padding-left: 10px;
23 }
24
25 ion-menu.md ion-list#inbox-list {
26   border-bottom: 1px solid var(--ion-background-color-step-150, #d7d8da);
27 }
28

```

Figura 6: Root (principal) Componentes de estilos

Descripción: Se da apariencia y estilo al sidebar y al navbar de la aplicación.

```

app.component.html
html > evaluacionlonic > src > app > app.component.html > ion-app > ion-split-pane > ion-menu > ion-content > ion-list#inbox-list
Go to component
1  <ion-app>
2    <ion-split-pane contentId="main-content">
3      <ion-menu contentId="main-content" type="overlay">
4        <ion-content>
5          <ion-list id="inbox-list">
6            <ion-list-header>Mi Evaluación</ion-list-header>
7            <ion-note>Ionic & Angular</ion-note>
8            <!-- Sidebar se muestran los links de la navegacion-->
9            @for (p of appPages; track p.title) {
10             <ion-menu-toggle auto-hide="false">
11               <ion-item routerDirection="root" [routerLink]="[p.url]" lines="none" detail="false"
12                 routerLinkActive="selected">
13                 <ion-icon aria-hidden="true" slot="start" [ios]="p.icon + '-outline'" [md]="p.icon + '-sharp'"></ion-
14                 <ion-label>{{ p.title }}</ion-label>
15               </ion-item>
16             </ion-menu-toggle>
17           }
18         </ion-list>
19       </ion-content>
20     </ion-menu>
21     <ion-router-outlet id="main-content"></ion-router-outlet>
22   </ion-split-pane>
23 </ion-app>

```

Figura 7: Sidebar Componentes HTML

Descripción: Se coloca la etiquetas o elementos para mostrar el contenido de el menu principal.

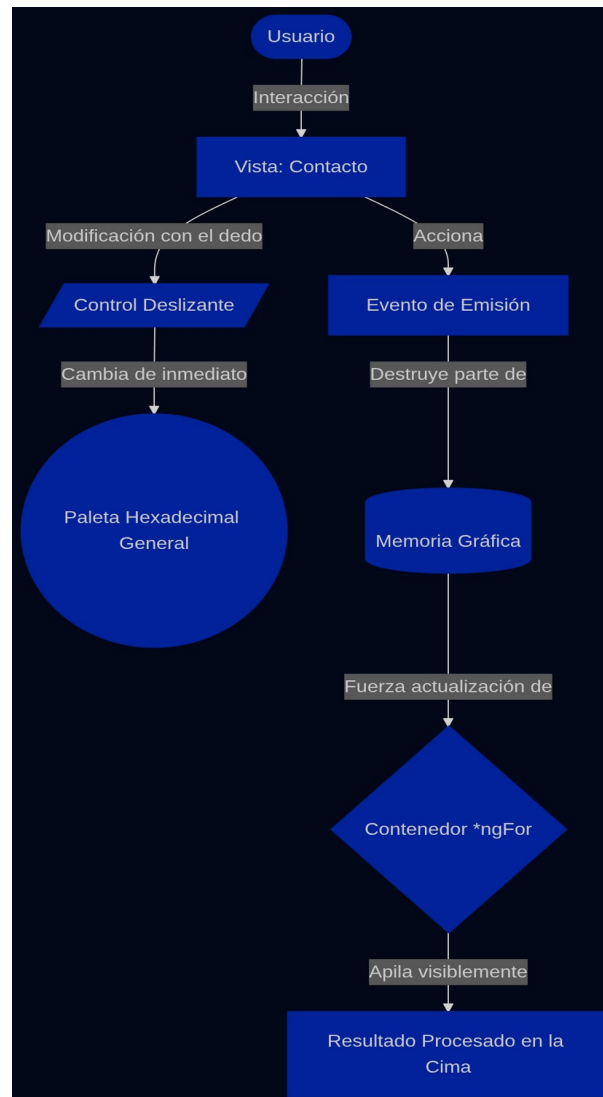


Figura 8: Diagrama del Comportamiento Tras Bambalinas

Descripción: Esquema lógico donde se expone la interacción del usuario en la vista "Contacto". Una modificación a través del dedo en el control deslizante cambia de inmediato la paleta hexadecimal general. Posteriormente, el evento de emisión destruye parte de la memoria gráfica, forzando la actualización del contenedor *ngFor y apilando visiblemente el resultado procesado en la cima.

APÉNDICE

Apéndice A. Fragmentos de acoplamiento que requieren refactorización (contacto.page.ts).

Algoritmo encargado de la traducción hexadecimal dinámica:

```
getColorPrioridad(nivel: number): string {  
  switch (nivel) {  
    case 1: return 'success';  
    case 2: return 'tertiary';  
    case 3: return 'primary';  
    case 4: return 'warning';  
    case 5: return 'danger';  
    default: return 'primary';  
  }  
}
```

Lógica responsable del almacenamiento volátil en donde la interfaz absorbe responsabilidades de bases de datos y comete el anti-patrón de uso de ventanas emergentes (alert):

```
enviarMensaje() {  
  if (this.nombre.trim() !== '' && this.mensaje.trim() !== '') {  
    this.mensajesEnviados.unshift({ nombre: this.nombre, mensaje: this.mensaje,  
prioridad: this.prioridad });  
  } else {  
    alert('Por favor, ingresa tu nombre y tu mensaje.');  }  
}
```